

Hospital Patient Vital Signs Monitoring

EC8203 Applied Big Data Engineering — Mini-Project Technical Report

Sep 30, 2026 · @Akila

Executive summary

This project builds an end-to-end **Lambda-architecture** data pipeline for a hospital ward that answers one business question: *which patients show concerning vital-sign trends right now, and how do yesterday's lab results change the risk picture for those patients going forward?* Bedside monitors for 20 synthetic patients stream a reading every 2 seconds into Kafka; a Spark Structured Streaming speed layer scores each patient in near real time (a simplified NEWS2 score) and raises alerts within seconds. Once a day, the pathology lab drops a CSV of results; an Airflow-orchestrated batch layer validates it, recomputes the previous day's vitals from an immutable Parquet lake, and produces a consolidated risk report showing exactly how each patient's risk tier changed once labs were factored in. Both views meet in one PostgreSQL database and are exposed through a FastAPI serving layer and two Grafana dashboards.

The system was built and verified as a 17-container Docker Compose stack by a team of three (platform/ingestion/observability, the Spark speed layer, and the Airflow batch layer/serving API). It was run continuously for soak tests exceeding 40 minutes and 9-43 simulated days, during which every one of 13 alert rules was deliberately triggered and confirmed to resolve, and the batch layer's daily reconciliation against the speed layer stayed below 1% discrepancy — direct, measured evidence for the Lambda design decision defended in Section 2.

Use case and business requirements

Use case 2 — Hospital Patient Vital Signs Monitoring. A hospital ward wants continuous, near-real-time monitoring of patient vitals from bedside sensors, correlated daily with lab test results the pathology lab uploads once a day. We interpreted the brief as five functional requirements and two non-functional ones.

#	Requirement	Where it is met
R1	Seconds-level awareness: a per-patient risk tier and alerts within one processing cycle of a reading	Speed layer (Spark Structured Streaming) writing <code>patient_status</code> and <code>alerts</code> every 5 seconds
R2	Trends, not only thresholds: deterioration shows as rising heart rate, falling SpO2 or falling blood pressure over minutes, not just single bad readings	2-minute sliding windows, per-patient trend slopes and a sustained-abnormal-window counter
R3	Robust daily lab ingestion: a late, missing, corrupt or repeated file must never corrupt the data	Airflow DAG with a file sensor, schema/row validation, quarantine, and an idempotent full-day reload
R4	Labs feed back into the live view: a patient with normal vitals but an abnormal lactate is still at risk	<code>patient_lab_risk</code> , re-read by the streaming job every micro-batch so the live tier updates within seconds of the batch layer committing
R5	A trustworthy daily consolidated report: recomputed from immutable data, ranked by risk, explaining what the labs changed, reproducible for any past day	Batch Spark job over the Parquet lake producing <code>patient_risk_report</code> plus an HTML/CSV report
NF1	Runs on a single laptop	Docker Compose, 17 containers, single-broker Kafka, single PostgreSQL instance, all with memory limits
NF2	Observable and replayable	Structured JSON logs, Prometheus metrics, 13 alert rules, and an idempotent design throughout so any day can be safely replayed

Simplifications stated up front (elaborated in Section 12): the patients are synthetic and generated by a seeded simulator; one simulated day compresses to 5 real minutes; and the risk score is a simplified adaptation of NEWS2 that omits respiration rate and consciousness level, so it is illustrative and **not clinically validated**.

Architecture decision: Lambda, not Kappa

We chose a **Lambda architecture**. The business question has two halves with genuinely different latency, replay and consistency requirements, and Lambda lets each half use the processing model that fits it, merging the results in one serving store.

Requirement	How Lambda meets it
Alerts within seconds	Speed layer: Spark Structured Streaming, 5-second micro-batches, event-time windows
Labs arrive as one daily file, sometimes late or corrected	Batch layer: an Airflow sensor, then validate then load; there is no need to force a file-based source through a log
The daily report must be correct, not approximate	The batch job recomputes the whole day from the immutable Parquet lake, including late data the speed layer's watermark dropped
Replay or backfill any past day	Re-run the batch job for <code>sim_day = N</code> , independent of Kafka's 24-hour retention
One scoring logic across both layers	<code>common/scoring.py</code> is imported by both the streaming job and the batch job, so the two views cannot silently diverge in their rules

Rejected alternative: Kappa. A single streaming codebase would process both vitals and labs (labs published to a compacted Kafka topic) and "reprocess" history by replaying the log. We rejected this for three reasons. First, reprocessing a day still needs the full log somewhere — either very long Kafka retention or a lake to replay from, which is exactly the batch storage Lambda already has, so Kappa does not remove that cost, it only hides it. Second, the daily report's natural semantics are "recompute and overwrite the partition for day N", which is simpler and safer to reason about as an idempotent batch job than as a long-running stateful streaming job with the same guarantee. Third, the lab file is batch by nature (one delivery a day, occasionally late or corrected), and the assignment's mandated orchestrator, Airflow, would have nothing meaningful to orchestrate under Kappa.

The cost of Lambda, paid honestly. Two code paths can drift apart. We mitigate this with the shared scoring module above, and — rather than assume the two views agree — we *measure* the disagreement every simulated day: the batch layer publishes `speed_batch_discrepancy_ratio`, comparing its exact per-window aggregates against the speed layer's windowed ones. A non-zero ratio is expected, because the speed layer deliberately trades completeness for latency (readings later than its 1-minute watermark

are excluded from windows, though they remain in the lake). Over a 43-simulated-day soak run this ratio averaged **0.54%** (min 0.13%, max 0.99%) — an order of magnitude below the 10% alert threshold — with zero windows the speed layer failed to produce and a mean heart-rate difference of at most 0.034 bpm per window. This is the strongest evidence for the architecture decision: the trade-off Lambda makes explicit is small, quantified, and continuously monitored rather than assumed away.

Ingestion and replay. The Kafka topic `vitals.raw` has 3 partitions keyed by `patient_id`, guaranteeing per-patient ordering (required for trend slopes) with enough parallelism for 20 patients. Retention is a short 24 hours because Kafka is a transport, not the system of record — replaying history reads the Parquet lake, which is exactly-once per reading and retained indefinitely, rather than the at-least-once topic. Kafka's own offsets are used only for a narrower kind of replay: resuming the speed layer from its checkpoint after a restart, which its short retention window is sized for.

Serving and consistency. The speed and batch views meet in one PostgreSQL database at two points with different consistency guarantees. Lab risk reaches the live view *eventually consistently*: the streaming job re-reads the newest `patient_lab_risk` row every micro-batch, so a patient's tier reflects yesterday's labs within about 5 seconds of the batch layer committing, and a quarantined lab file leaves the previous lab risk in force rather than dropping it to zero. The daily report is *strongly consistent* by construction: it is computed from the immutable lake, not the speed tables, so late data and duplicates are handled correctly and replaying a day reproduces an identical report. The API exposes the freshness of each side (data age, latest report day, latest lab day) so a reader can see how current each part of the answer is.

Architecture diagram

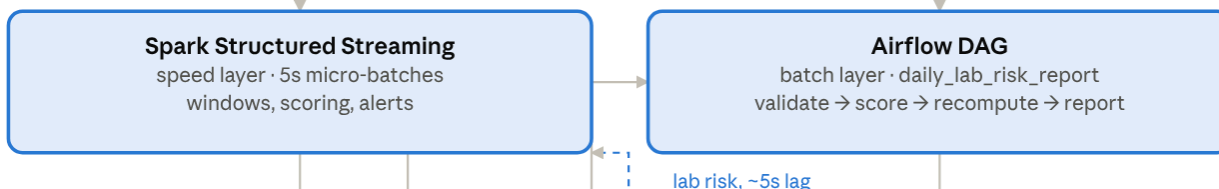
Two processing paths meet in one PostgreSQL database

Solid: data flow. Dashed: the batch layer's lab risk read back into the speed layer.

INGESTION



PROCESSING



STORAGE



SERVING



system architecture · ingestion, processing, storage, serving

Data flows top to bottom through the four layers the assignment asks for. The one dashed arrow is the detail that makes the business question answerable: the speed layer re-reads `patient_lab_risk` every micro-batch, so a lab result the batch layer just computed changes a patient's live risk tier within seconds, not the next day.

Technology stack

Every choice below is justified against this use case's constraints, not general popularity.

Layer	Choice	Why this use case needs it
Ingestion	Kafka (KRaft, 1 broker), topic <code>vitals.raw</code> with 3 partitions keyed by <code>patient_id</code> , plus <code>vitals.dlq</code>	Per-patient ordering is a correctness requirement for trend slopes, not a nice-to-have. A log decouples 20 independent bedside monitors from a speed layer that may be temporarily down and gives it something to resume from after a restart.
Stream processing	Spark Structured Streaming	The business question needs event-time windows and watermarks over deliberately out-of-order sensor data, plus a stream-static join against daily lab risk that must pick up new rows without a restart — both native to Structured Streaming. Apache Storm was rejected: it would need hand-rolled state management to reach the same windowing and join semantics this project gets natively.
Orchestration	Apache Airflow, LocalExecutor	The lab side is a genuine multi-step batch dependency chain (wait for file, validate, load, score, recompute, report, health-check, archive) with retries and a visible run history — exactly Airflow's job. The DAG is the entire batch half of the Lambda architecture, not a token integration.

Layer	Choice	Why this use case needs it
Storage / serving	PostgreSQL for speed and batch tables; Parquet on a local volume (standing in for S3/HDFS) for the lake	Postgres: the write pattern is small, frequent upserts (about 20 patient rows plus about 100 window rows every 5 seconds) with ad-hoc SQL joins across speed and batch tables — a poor fit for Cassandra's append-oriented, join-averse model at this scale. Parquet: the lake must be cheaply re-scannable by simulated day for the batch recompute and for replay; a columnar, partition-pruned format is the natural fit, and the brief explicitly allows a file system with Parquet in place of a database for this role.
Serving API	FastAPI	Read-only endpoints over both layers' tables, OpenAPI documentation generated from the response models so the docs cannot drift from the code, and a natural home for <code>/health</code> and <code>/metrics</code> .
Observability	Prometheus, Alertmanager, Grafana, structured JSON logs	Every component here is a different language and runtime (Python, JVM, Airflow); Prometheus's pull model plus one shared metric-naming contract let a single dashboard and a single alert-rule set cover all of them without any component needing to know about the others.
Packaging	Docker Compose, one file per layer joined by <code>include:</code>	Three team members each own a Compose fragment (<code>compose/spark.yml</code> , <code>compose/airflow.yml</code> , <code>compose/api.yml</code>) without ever touching the same file — a direct, practical answer to three people needing to work in parallel, not a generic default.

Data ingestion implementation

Two simulated sources, both plain Python processes with no framework, so every number in the report can be traced back to a line of generator code.

Vitals stream (`simulators/vitals_producer.py`): 20 synthetic patients, each with a physiological baseline (heart rate, SpO2, systolic/diastolic blood pressure, temperature) drawn once from a seeded cohort and perturbed every 2 seconds by mean-reverting noise. Four scripted patients carry a deterioration episode (sepsis-like or respiratory) that ramps, holds and recovers; two further "occult" patients keep normal vitals throughout but their labs turn abnormal from a fixed simulated day on. This scripted, known-in-advance ground truth (`data/ground_truth/episodes.json`) is what makes it possible to check that alerts, the lab-risk feedback loop and the daily report catch the right patients, rather than merely running without crashing.

Lab file (`simulators/lab_generator.py`): one CSV per simulated day (`labs_day_N.csv`), correlated with the same scripted story — abnormal lactate, white cell count and CRP while a sepsis episode is active; abnormal creatinine, potassium or glucose for chronic conditions; abnormal results for the occult patients from their trigger day. Files are written to a temporary path and atomically renamed into place, so a consumer can never observe a half-written file.

Robustness and fault injection. The Kafka producer is idempotent (`acks=all` , unlimited retries) so a broker-side retry cannot itself create the duplicates the pipeline is built to tolerate. A configurable fault injector (`FAULT_PROFILE=off|low|chaos`) deliberately introduces null vitals, physiologically impossible values, duplicate event IDs, events held back 20-90 seconds past their timestamp, and short sensor dropouts — not as noise, but so that validation, deduplication, watermark handling and alerting have something real to prove themselves against (Section 8 shows the results of exercising this). Both sources derive `sim_day` from one shared, race-free simulated clock (a single epoch agreed by whichever process starts first, verified under concurrent start-up in unit tests), and every random choice is seeded, so a given seed always reproduces the same cohort, episodes and lab values.

Processing layer

Speed layer (Spark Structured Streaming)

Two streaming queries read `vitals.raw` , each with its own checkpoint so they recover independently:

1. **vitals_readings** (5-second trigger): parses each record against an explicit schema, marks (never silently drops) the first validation rule it fails as a `rejection_reason`, joins against the `patients` dimension and the newest `patient_lab_risk` row (a stream-static join re-planned every micro-batch, which is how a lab result reaches the live view without restarting the job), computes MAP, pulse pressure, the NEWS sub-scores and the total risk tier, then de-duplicates on `event_id` within a watermark. Valid rows are written to the Parquet lake and to `patient_status`; invalid rows go to `vitals.dlq` and `dlq_events`.
2. **vitals_windows** (5-second trigger, update mode): the valid stream is watermarked at 1 minute, de-duplicated again, and aggregated into 2-minute windows sliding every 30 seconds (scaled down from the plan's 5-minute reference window to match the compressed simulated clock). Trend slopes and a sustained-abnormal-window counter are computed in the driver over the small, bounded set of recent windows per patient.

The risk score is a simplified NEWS2 adaptation (sub-scores for heart rate, SpO2, systolic blood pressure and temperature) plus lab points capped at 4, giving four tiers: LOW, MEDIUM, HIGH, CRITICAL. It is shared with the batch layer through one module, `common/scoring.py`, so the two layers cannot silently apply different rules. Alerts (reason codes such as `TOTAL_SCORE_HIGH`, `SUSTAINED_ABNORMAL`, `WORSENING_TREND`) follow an open-to-resolved lifecycle with deterministic IDs, so replaying a micro-batch re-derives the same alerts rather than duplicating them.

Batch layer (Airflow)

The DAG `daily_lab_risk_report` runs once per simulated day: a file sensor waits for the day's lab CSV, `validate_lab_file` checks schema, types, reference ranges and duplicate or unknown patients (quarantining bad rows or the whole file rather than failing the run), `load_lab_results` replaces that day's rows in one transaction, `compute_lab_risk` derives lab points into `patient_lab_risk` (immediately visible to the speed layer above), `batch_vitals_job` is a separate Spark job in local mode that recomputes the previous day's vitals from the immutable lake and reconciles them against the speed layer's own windows, `build_risk_report` produces `patient_risk_report` plus an HTML/CSV report ranked by risk with an explicit before-labs/after-labs comparison, and a health check and archive step close out the run. Every task writes a row to `pipeline_run_log`, and every failure increments a Prometheus counter and triggers the matching alert (Section 6).

Both queries and the DAG are idempotent by design: a day can be replayed (`make replay-day DAY=N`) and produces byte-identical downstream rows, verified in the test suite and confirmed live by killing and restarting both the streaming job and the Airflow scheduler mid-run (Section 8).

Storage and serving layer

Store	Content	Key / layout	Why
Parquet lake	Every valid reading plus event time, MAP, pulse pressure, NEWS score and Kafka coordinates	data/lake/vitals/sim_day=N/part-<checkpoint>-b<batch>-NNN.parquet	Immutable master dataset for the batch layer; columnar and partition-pruned by day; file names include the checkpoint and micro-batch id so a replay overwrites only its own files, never duplicating or clobbering older ones
vitals_window	Per patient, per 2-minute window: average/min/max of five vitals, count, window NEWS, trend slopes	Primary key (patient_id, window_start)	Upsert target of the update-mode streaming query; feeds the ward dashboard's time-series charts
patient_status	One row per patient: latest vitals, scores, lab points, risk tier, trend flag	Primary key patient_id	The "right now" ward view in O(patients) time; a guarded upsert ensures last_reading_at never moves backwards
alerts	Alert open/resolved lifecycle	Primary key alert_id (deterministic UUID), unique open alert per patient and reason	Replay-safe: re-processing a batch cannot open a duplicate alert
lab_results, patient_lab_risk, patient_risk_report, batch_vitals_daily, speed_batch_reconciliation	Daily batch outputs	Keyed by simulated day and patient	Written by the Airflow DAG; patient_risk_report is what the consolidated daily report is rendered from

PostgreSQL was chosen over Cassandra because the write pattern here is small, frequent upserts with ad-hoc SQL joins between the speed and batch tables — exactly what Postgres is good at and Cassandra is not. The **serving layer** is a FastAPI application with thirteen read-only endpoints in five groups: ward (/api/ward/summary merges both layers in one response), patients (paginated, sorted by risk, with a detail view joining live status to the newest lab risk), alerts, reports (JSON and rendered HTML for any past day), and operations (/health returns a 503 when the newest reading is older than 60 seconds, /metrics exposes request latency, data freshness, patients per tier and open alerts per severity). The API holds a small pool of read-only, time-limited transactions, so a slow query cannot pin a connection, and a database outage returns a clean 503 instead of a stack trace. Two Grafana dashboards (Pipeline Health over Prometheus, Ward Live Monitoring over PostgreSQL) are provisioned as code and require no manual setup after docker compose up .

Observability design

What is measured and why:

Signal	Instrument	Question it answers
Is data still arriving?	<code>simulator_last_emit_timestamp_seconds</code> , Kafka topic offsets, <code>pipeline_last_event_age_seconds</code>	Has the pipeline gone silent?
Is the speed layer keeping up?	<code>kafka_consumergroup_lag</code> , <code>spark_batch_duration_seconds</code>	Is processing falling behind ingestion?
Is data quality holding?	<code>vitals_valid_total</code> , <code>vitals_dlq_total{reason}</code>	What fraction of readings are rejected, and why?
Is the batch layer on schedule?	<code>airflow_dag_last_success_timestamp{dag_id}</code> , <code>lab_file_missing_total</code>	Did today's report actually get produced?
Do the two Lambda views agree?	<code>speed_batch_discrepancy_ratio</code>	Is the architecture's core trade-off staying small?
Is the ward-facing surface healthy?	<code>api_request_duration_seconds</code> , <code>/health</code>	Does the API respond, and is its data fresh?

Structured logging. Every service, Python or JVM, writes one JSON object per stdout line through a shared formatter: `{ts, level, service, stage, event, run_id, ...}`, where `stage` is one of ingestion, processing, storage, serving, orchestration or observability. This lets a single `docker compose logs | grep` isolate one pipeline stage across every container with no service-specific knowledge, and every line carries a correlation id for one container's lifetime.

Thirteen alert rules, written as Prometheus rules-as-code and unit-tested against synthetic time series with `promtool` before the components they depend on even existed — several rules reference metrics owned by the other two team members (`kafka_consumergroup_lag{consumergroup="spark-speed-layer"}`, `airflow_dag_last_success_timestamp`), and having them verified in advance turned integration into confirming names rather than debugging alert logic under time pressure.

Alert	Trigger condition
<code>NoVitalsData</code>	No Kafka offset growth on <code>vitals.raw</code> for 45 seconds
<code>SimulatorDown</code> / <code>StreamingQueryStopped</code> / <code>ApiDown</code> / <code>ScrapeTargetDown</code>	The corresponding component stops answering its metrics endpoint
<code>ProducerDeliveryErrors</code>	Any sustained Kafka delivery failure rate
<code>DlqRateHigh</code>	More than 5% of readings rejected by validation over 2 minutes
<code>ConsumerLagHigh</code>	The speed layer's consumer group falls more than 500 messages behind
<code>PipelineDataStale</code>	Newest processed reading older than 60 seconds
<code>SparkBatchSlow</code>	Micro-batches sustained above 10 seconds against a 5-10 second trigger
<code>LabFileMissing</code>	The daily file sensor times out
<code>DagFailed</code>	No successful DAG run in over 11 minutes (two missed simulated days)
<code>SpeedBatchDiscrepancy</code>	Speed-versus-batch reconciliation exceeds 10%

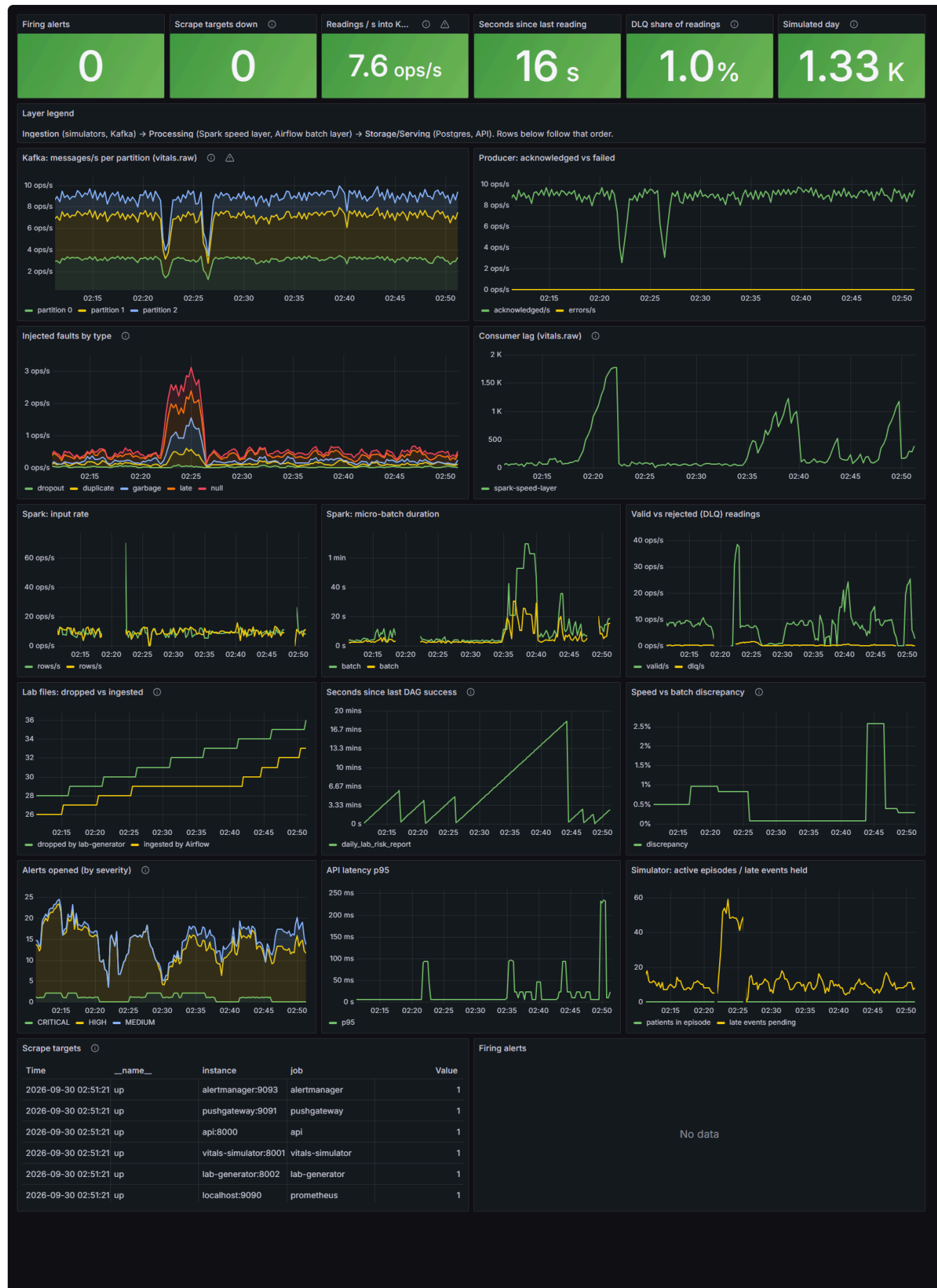
Alerts route through Alertmanager to a small webhook that logs every firing and resolved transition as a structured line and appends it to a durable file (`data/alerts/alerts.jsonl`), independent of Prometheus's own retention — the evidence trail behind every claim in Section 8. A deliberate design choice: `NoVitalsData` uses an `absent()` guard because "no data at all" is itself the failure, but `DagFailed` does not, because before the very first DAG success there is no time series to be absent, and guarding it would open a false alert on every fresh `docker compose up` while Airflow initialises.

Results

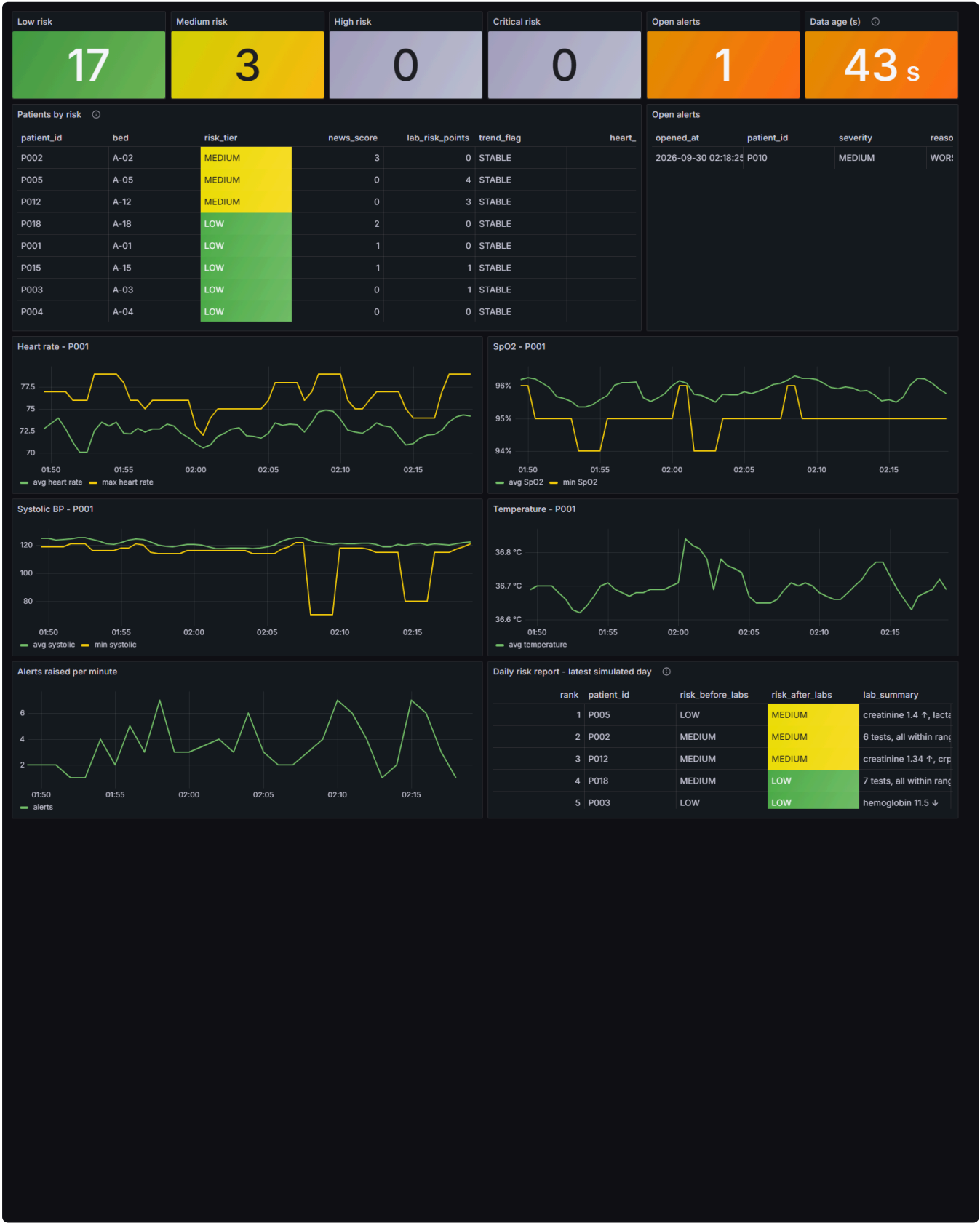
All figures below come from the live, full 17-container stack, not a mock-up.

Pipeline health dashboard, captured across a 40-minute window that deliberately included every fault demonstration in this report (a Spark restart, a chaos-level fault injection, and an 11-minute Airflow outage): the Kafka throughput dip and recovery, the

DLQ spike during `DlqRateHigh`, the consumer-lag spike and drain, the micro-batch slowdown during resource contention, and the "seconds since last DAG success" sawtooth climbing to its `DagFailed` peak before resetting on recovery. By the end of the window every metric is back at baseline and 0 alerts are firing.



Ward live monitoring dashboard: current risk tiers across the 20-patient cohort, open alerts, per-patient vital-sign trends, and, critically for this use case, the daily risk report table showing patients whose tier changed once labs were applied (P005 and P012 rising to MEDIUM on abnormal lab points, P018 dropping back to LOW).



Heart rate - P001

SpO2 - P001

Systolic BP - P001

Temperature - P001

Alerts raised per minute

Daily risk report - latest simulated day

rank	patient_id	risk_before_labs	risk_after_labs	lab_summary
1	P005	LOW	MEDIUM	creatinine 1.4 ↑, lactate 1.5 ↑
2	P002	MEDIUM	MEDIUM	6 tests, all within range
3	P012	MEDIUM	MEDIUM	creatinine 1.34 ↑, crp 1.2 ↑
4	P018	MEDIUM	LOW	7 tests, all within range
5	P003	LOW	LOW	hemoglobin 11.5 ↓

Sample consolidated daily report (rendered HTML, one simulated day): patients ranked by risk after labs, with an explicit risk-before-labs versus risk-after-labs comparison, the artefact that most directly answers the assignment's business question.

Daily consolidated risk report - simulated day 42

Vitals of day 41 (batch recompute from the Parquet lake) combined with the lab file labs_day_042.csv (labs collected on day 41). Generated 2026-09-28 18:35:44 UTC. Scores: simplified NEWS2 adaptation, not clinically validated.

Patients
20

Risk after labs
Low 17 · Medium 3 · High 0 · Critical 0

Tier changed by labs
1 (1 up, 0 down)

Speed vs batch discrepancy
0.36% of readings

How yesterday's labs change the risk picture

- P007: LOW → MEDIUM (lab points 0 → 3: creatinine:high, potassium:high)

All patients, ranked by risk after labs

#	Patient	Risk before labs	Risk after labs	End-of-day NEWS	Lab points	Trend	HR (day)	SpO2 (day)	Vitals summary	Lab flags	Alerts
1	P005 Synthetic Patient 005 - bed A-05	MEDIUM	MEDIUM	0	4 → 4	STABLE	 66-76	 92-98	HR 71 (65-77) · SpO2 97 (85-98) · SBP 127 (73-133) · T 36.8 · peak NEWS 3	creatinine 1.39 ↑, lactate 2.2 ↑, wbc 11.7 ↑	4
2	P012 Synthetic Patient 012 - bed A-12	MEDIUM	MEDIUM	0	4 → 4	STABLE	 66-74	 96-98	HR 70 (65-75) · SpO2 97 (95-98) · SBP 134 (127-182) · T 36.4 · peak NEWS 1	creatinine 1.34 ↑, lactate 2.2 ↑, wbc 11.8 ↑	1
3	P007 Synthetic Patient 007 - bed A-07	LOW	MEDIUM ▲	0	0 → 3	STABLE	 78-92	 97-99	HR 84 (77-93) · SpO2 99 (88-100) · SBP 125 (119-174) · T 36.7 · peak NEWS 3	creatinine 1.43 ↑, potassium 5.3 ↑	1
4	P002 Synthetic Patient 002 - bed A-02	LOW	LOW	2	0 → 0	STABLE	 81-87	 92-94	HR 84 (80-88) · SpO2 93 (92-94) · SBP 116 (109-167) · T 36.5 · peak NEWS 3	7 tests, all within range	0
5	P018 Synthetic Patient 018 - bed A-18	LOW	LOW	2	0 → 0	STABLE	 65-74	 93-96	HR 68 (64-75) · SpO2 95 (84-96) · SBP 107 (65-114) · T 36.7 · peak NEWS 4	7 tests, all within range	3
6	P015 Synthetic Patient 015 - bed A-15	LOW	LOW	1	1 → 1	STABLE	 85-108	 95-98	HR 91 (84-155) · SpO2 97 (95-98) · SBP 125 (85-132) · T 36.8 · peak NEWS 3	glucose 155 ↑	2
7	P003 Synthetic Patient 003 - bed A-03	LOW	LOW	0	1 → 2	STABLE	 73-113	 97-99	HR 79 (71-147) · SpO2 98 (97-99) · SBP 126 (119-182) · T 36.7 · peak NEWS 3	crp 14.8 ↑, hemoglobin 11.4 ↓	1
8	P014 Synthetic Patient 014 - bed A-14	LOW	LOW	0	1 → 1	STABLE	 71-82	 97-99	HR 76 (70-82) · SpO2 98 (97-99) · SBP 124 (81-176) · T 36.9 · peak NEWS 3	glucose 61 ↓	1
9	P001 Synthetic Patient 001 - bed A-01	LOW	LOW	0	0 → 0	STABLE	 69-80	 95-97	HR 73 (68-81) · SpO2 96 (95-97) · SBP 121 (114-128) · T 36.7 · peak NEWS 1	6 tests, all within range	0
10	P004 Synthetic Patient 004 - bed A-04	LOW	LOW	0	0 → 0	STABLE	 76-108	 97-98	HR 82 (74-147) · SpO2 97 (96-98) · SBP 116 (110-122) · T 36.7 · peak NEWS 3	4 tests, all within range	1
11	P006 Synthetic Patient 006 - bed A-06	LOW	LOW	0	0 → 0	STABLE	 65-74	 97-99	HR 69 (64-75) · SpO2 98 (97-99) · SBP 121 (116-125) · T 36.4 · peak NEWS 0	5 tests, all within range	0
12	P008 Synthetic Patient 008 - bed A-08	LOW	LOW	0	0 → 0	STABLE	 77-85	 95-97	HR 81 (75-86) · SpO2 96 (95-97) · SBP 122 (118-129) · T 36.7 · peak NEWS 1	6 tests, all within range	0
13	P009 Synthetic Patient 009 - bed A-09	LOW	LOW	0	2 → 0	STABLE	 71-78	 95-99	HR 76 (70-79) · SpO2 98 (89-99) · SBP 129 (87-139) · T 36.8 · peak NEWS 3	7 tests, all within range	4
14	P010 Synthetic Patient 010 - bed A-10	LOW	LOW	0	0 → 0	STABLE	 72-79	 97-99	HR 75 (70-80) · SpO2 98 (97-99) · SBP 146 (96-159) · T 36.7 · peak NEWS 2	6 tests, all within range	0
15	P011 Synthetic Patient 011 - bed A-11	LOW	LOW	0	0 → 0	STABLE	 65-73	 98-100	HR 69 (64-73) · SpO2 99 (98-100) · SBP 113 (71-118) · T 36.6 · peak NEWS 3	7 tests, all within range	2
16	P013 Synthetic Patient 013 - bed A-13	LOW	LOW	0	0 → 0	STABLE	 70-96	 96-98	HR 74 (69-141) · SpO2 98 (88-99) · SBP 136 (129-142) · T 36.9 · peak NEWS 3	6 tests, all within range	2
17	P016 Synthetic Patient 016 - bed A-16	LOW	LOW	0	1 → 0	STABLE	 79-111	 97-100	HR 83 (77-142) · SpO2 99 (90-100) · SBP 132 (125-188) · T 36.2 · peak NEWS 3	4 tests, all within range	3
18	P017 Synthetic Patient 017 - bed A-17	LOW	LOW	0	0 → 0	STABLE	 66-74	 98-99	HR 70 (66-76) · SpO2 98 (97-99) · SBP 114 (74-122) · T 36.8 · peak NEWS 3	6 tests, all within range	1
19	P019 Synthetic Patient 019 - bed A-19	LOW	LOW	0	0 → 0	STABLE	 68-74	 96-98	HR 72 (66-75) · SpO2 97 (95-98) · SBP 120 (115-125) · T 36.6 · peak NEWS 1	5 tests, all within range	0
20	P020 Synthetic Patient 020 - bed A-20	LOW	LOW	0	0 → 0	STABLE	 82-90	 94-98	HR 85 (80-91) · SpO2 98 (90-99) · SBP 136 (128-140) · T 37.0 · peak NEWS 3	7 tests, all within range	1

Sample API output (`/api/ward/summary`), combining live speed-layer state with the latest batch-layer report and reconciliation in one response:

```
$ curl http://localhost:8000/api/ward/summary

$ curl -s http://localhost:8000/api/ward/summary | jq
{
  "generated_at": "2026-09-29T05:42:11.900161Z",
  "total_patients": 20,
  "patients_with_data": 20,
  "patients_by_tier": {
    "LOW": 15,
    "MEDIUM": 5,
    "HIGH": 0,
    "CRITICAL": 0
  },
  "active_alerts": 1,
  "active_alerts_by_severity": {
    "MEDIUM": 0,
    "HIGH": 1,
    "CRITICAL": 0
  },
  "avg_vitals": {
    "heart_rate": 76.2,
    "spo2": 96.8,
    "systolic_bp": 126.9,
    "diastolic_bp": 78.4,
    "temperature": 36.7,
    "news_score": 0.4
  },
  "readings_per_minute": 555.0,
  "freshness": {
    "last_reading_at": "2026-09-29T05:41:55.895000Z",
    "data_age_seconds": 16.0,
    "stale": false,
    "max_age_seconds": 60.0,
    "current_sim_day": 175
  },
  "batch": {
    "latest_report_sim_day": 174,
    "latest_lab_sim_day": 175,
    "last_dag_success_at": "2026-09-29T05:39:40.517531Z",
    "last_discrepancy_ratio": 0.005564
  },
  "concerning_patients": []
}
```

Measured soak-run results.

Metric	Platform layer alone (2026-09-25)	Full stack, demos included (2026-09-29/30)
Duration / simulated days	30.4 min / 7 days	40.5 min / 9 days
Kafka throughput (msgs/s)	min 8.1, avg 9.2, max 9.7	min 3.5 (during chaos-fault demo), avg 8.9, max 9.6
Lab files landed	7, none missing	46, none missing
Peak memory, all containers	779 MiB	3990 MiB

Metric	Platform layer alone (2026-09-25)	Full stack, demos included (2026-09-29/30)
Alerts left firing at the end	0 (aside from B and C services not yet deployed)	0
Speed-vs-batch discrepancy (separate 43-day run)	not applicable	mean 0.54%, max 0.99%, versus a 10% alert threshold

All 13 alert rules were confirmed firing live at least once, and every one resolved once the underlying condition was fixed; the full narrative, with exact log lines and timings, is in `docs/platform_observability.md` section 3.4 and `docs/batch_serving.md` section 4.3 of the codebase. `python scripts/e2e_smoke.py --full` runs 10 checks spanning ingestion, the speed layer, the batch layer and the API; it passed both immediately before and immediately after the full-stack soak run, confirming the system recovers cleanly from every deliberate failure injected into it.

Limitations, trade-offs, and production-scale changes

Scoring. The risk score is a simplified adaptation of NEWS2 without respiration rate or consciousness level, and it is not clinically validated. Single-vital alerts fire on every random spike by design (the brief's own rule: any single vital scoring 3 triggers at least MEDIUM), which in a real deployment would need persistence, for example two abnormal readings out of three, to avoid alert fatigue.

Time compression. One simulated day is 5 real minutes, so clinical time windows (a 2-minute trend window, an 11-minute DAG-failure threshold) are scaled down and not clinically calibrated; at real time scales the batch job would have a full day, not 5 minutes, to run.

Single points of failure by design, for a laptop-sized system. One Kafka broker with replication factor 1, one PostgreSQL instance serving both the application and Airflow's metadata, and the batch Spark job running in local mode inside the Airflow scheduler rather than on a cluster. At production scale: a multi-broker Kafka cluster with proper replication and a schema registry; the Parquet lake moved to object storage with a table format such as Delta or Iceberg for atomic commits and compaction of small per-batch files; the batch Spark job submitted to a real cluster (for example via `SparkSubmitOperator`); separate databases for the application and for Airflow's metadata, with read replicas for the API.

No security layer. The API has no authentication, and inter-service traffic is unencrypted. Real patient data would require authentication, role-based access control, audit logging, TLS everywhere, and the daily report files moved from a shared folder to access-

controlled storage.

Reconciliation is partial. The speed-versus-batch discrepancy metric compares window counts and mean heart rate only; a production version would compare every vital sign and the NEWS score per window, and would separate genuine late-data discrepancy from a logic drift between the two scoring implementations.

Operational rough edges found and accepted rather than fixed, each a candidate for a follow-up iteration: the Airflow scheduler's default 10-second stop grace period is shorter than the JVM needs to shut down cleanly, so a `docker compose stop` on it currently ends in a SIGKILL rather than a graceful exit (harmless here because every write is transactional, but worth a longer `stop_grace_period` in production); catch-up after a scheduler outage is automatic for the current interval but relies on `make replay-day` for anything missed while Airflow itself was down, since the DAG runs with `catchup=False`.

Assumptions and simplifications

- Time is compressed 288 times: one simulated day equals 5 real minutes (`SIM_DAY_SECONDS=300`). All timestamps are UTC.
- 20 synthetic patients on one ward: 4 with a scripted deterioration episode, 2 "occult" (normal vitals, abnormal labs), the rest stable with baseline noise and occasional random spikes.
- All data is synthetic, generated by a seeded simulator; no real patient information is used anywhere.
- The risk score is a simplified NEWS2 adaptation for demonstration and is not clinically validated.
- A single Kafka broker (replication factor 1), a single PostgreSQL instance, and no authentication or TLS, all acceptable for a mini-project but not for production.
- Parquet on a local Docker volume stands in for the S3 or HDFS object storage a production deployment would use.
- Where the assignment left scope open, we added fields and detail (for example, extra Postgres columns beyond the minimum contract, additional Prometheus metrics) rather than removed anything the brief specified.

Individual contributions

Member	Role	Delivered
Akila Piumantha (A)	Platform, ingestion and observability	Docker Compose for the whole stack; both simulated sources with fault injection; Kafka topic design; Prometheus, Alertmanager, Grafana and all 13 alert rules with promtool unit tests; the CI pipeline; the end-to-end smoke test and soak-run tooling; this report's platform, ingestion and observability chapters
Member B (name to be confirmed)	Speed layer	The Spark Structured Streaming job (both queries), shared scoring module, the Parquet lake writer, the alert engine, performance tuning, and the architecture-decision and processing-layer chapters this report draws on
Layanga Rajapakshe (C)	Batch layer, serving and reporting	The Airflow DAG and batch Spark job, the PostgreSQL schema, the daily consolidated risk report, the FastAPI serving layer, and the use-case, serving-layer and results material this report draws on

Detailed task-level ownership and completion evidence for all three members is tracked in `TASK_BOARD.md` in the project repository.